

Table of contents

Recherche Tabu (Tabu Search)	2
Principes de Base de la Recherche Tabu	2
Introduction et idée générale	2
Principes de base (2/3)	2
Principes de base (3/3)	3
Pourquoi la liste tabu ne peut-elle pas être permanente ?	3
Diagramme de flux de la Recherche Tabu (TS)	5
Exemple (1/4) : Approche par force brute	6
Exemple (2/4) : Recherche aléatoire (Random Search)	6
Exemple (3/4) : Recherche Tabu (TS)	6
Exemple (4/4) : Recherche Tabu (TS)	6
Convergence de TS (1/3)	7
Convergence de TS (2/3)	7
Convergence de TS (3/3) : Illustration	8
Liste Tabu (a.k.a. Liste d'Exclusion)	8
Objectifs et construction	8
Plus sur les mouvements interdits (1/2)	8
Plus sur les mouvements interdits (2/2)	9
Mise à jour et gestion de la liste tabu	9
Mémoire à court terme de taille limitée M	9
Mémoire à court terme avec temps d'interdiction k	10
Exemple de mémoire à court terme (1/2)	10
Exemple de mémoire à court terme (2/2)	11
Mémoire à long terme (Long-term Memory)	11
Utilisation de la mémoire	12
Exploration vs. Exploitation	12
Problème d'Affectation Quadratique (QAP)	12
Définition du QAP (1/2)	12
Définition du QAP (2/2)	13
Exemple : Minimiser la longueur des connexions	13
Exemple avec 4 objets et 4 emplacements	14
Matrices de flux et de distance	14
Code pour calculer la fitness QAP	15
Solution par Tabu Search	16
Voisinages standards	16
Évaluation du voisinage	17
Liste tabu et mémoire à court terme	17
Implémentation de la liste tabu (1/2)	17
Implémentation de la liste tabu (2/2)	18
Le problème de référence NUG5	18

Application de Tabu Search sur NUG5	18
Itération #1	19
Liste tabu à l'itération #1 (1/2)	19
Liste tabu à l'itération #1 (2/2)	20
Itération #2	20
Liste tabu à l'itération #2	20
Itération #3	21
Liste tabu à l'itération #3	21
Itération #4	22
Liste tabu à l'itération #4	22
Itérations suivantes	23

Recherche Tabu (Tabu Search)

Principes de Base de la Recherche Tabu

Introduction et idée générale

Introduite par **Fred Glover en 1986**.

Idée générale :

→ Conçue pour guider les recherches locales au-delà des optima locaux en évitant de revenir vers des solutions précédemment visitées.

La recherche Tabu (TS) :

- Est essentiellement **non-markovienne**
- S'appuie sur les méthodes de recherche locale (comme le **hill climbing**), en se déplaçant itérativement d'une solution vers une meilleure solution voisine
- Utilise une **structure de mémoire** pour : (i) échapper aux optima locaux et (ii) éviter de revisiter les solutions récemment explorées
- Explore l'espace de recherche de manière plus large et "intelligente"

Principes de base (2/3)

Bien adaptée aux problèmes d'optimisation combinatoire, tels que les **quadratic assignment problems** (QAPs).

Comme la plupart des métaheuristiques, les principes fondamentaux sont plutôt simples :

- Explorer l'espace de recherche de voisin en voisin : $x_n \rightarrow x_{n+1} \in V(x_n)$
- L'opération d'exploration cherche le "meilleur" $x_{n+1} \in V(x_n)$

- “Meilleur” signifie :
 - Solution non-tabu
 - Fitness optimale dans $V(x_n) \setminus \{x_n\}$ (indépendamment de la valeur de $f(x_n)$, la fitness peut être dégradée)
 - Si plusieurs points donnent la même fitness optimale : sélection aléatoire

Principes de base (3/3)

Spécificité clé :

→ Existence de points ou mouvements **interdits**, ou **tabu**.

Objectif de cette liste tabu :

→ Empêcher l’opérateur de recherche de revenir vers des configurations précédemment explorées.

L’attribut tabu n’est pas permanent :

→ Les points/mouvements interdits sont stockés dans une **mémoire à court terme**.

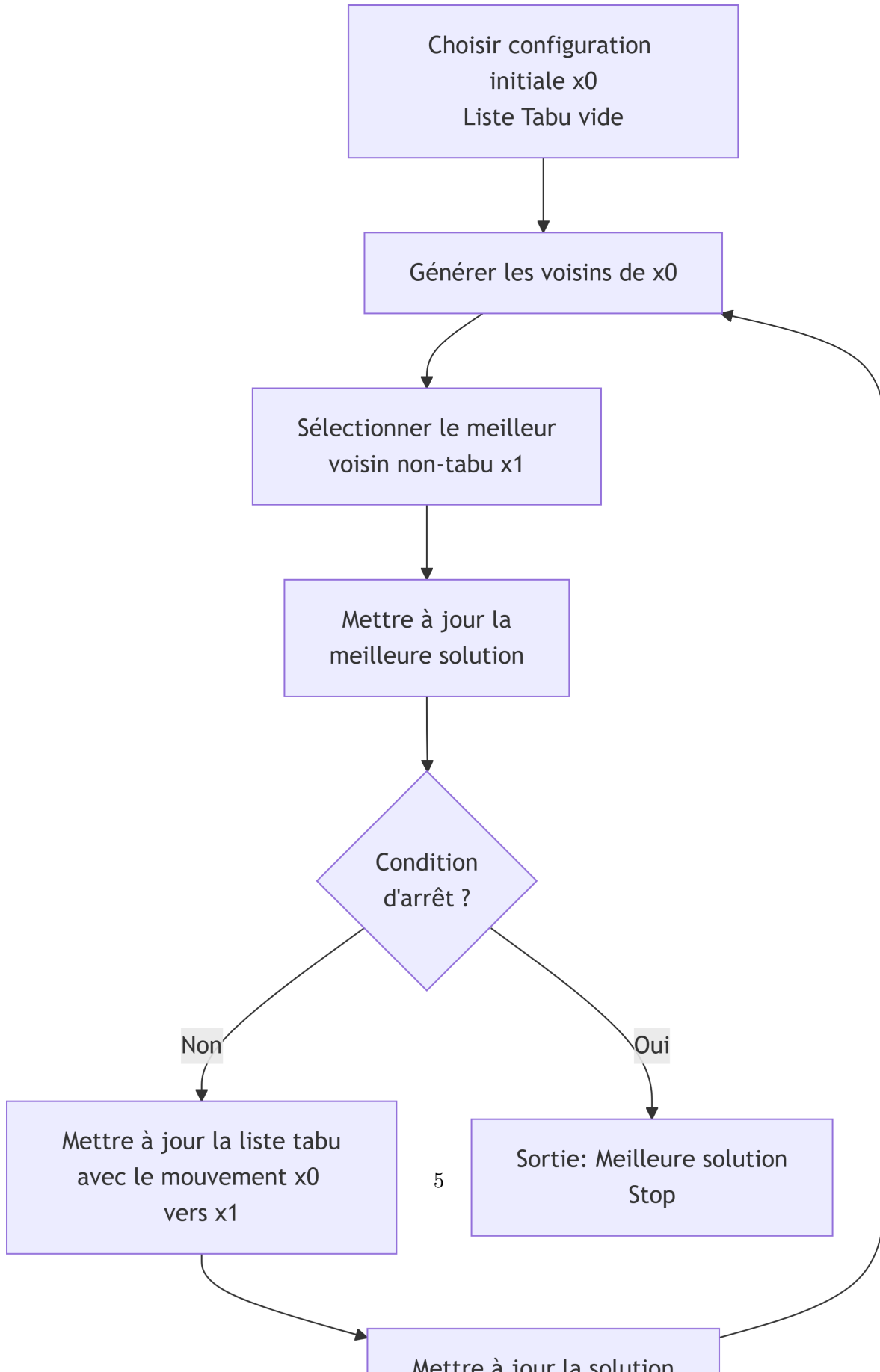
→ La méthode tabu opère sur la base d’une **mise à jour constante** de la liste tabu pendant le processus d’exploration.

Pourquoi la liste tabu ne peut-elle pas être permanente ?

Comme montré ci-dessous (marcheur dans $\mathbb{Z}^2$ avec voisinage NEWS), la liste tabu pourrait bloquer le processus si elle était permanente.

En effet, si on interdit définitivement tous les points visités, on peut se retrouver dans une situation où tous les voisins du point actuel sont interdits, bloquant ainsi la progression de l’algorithme.

Diagramme de flux de la Recherche Tabu (TS)



→ L'élément central est la **gestion de la liste tabu**.

Exemple (1/4) : Approche par force brute

Deux extrema, dans un domaine de taille 20×20 : en $(6, 6)$ et $(10, 10)$, respectivement. Celui en $(10, 10)$ est le max global.

Une recherche exhaustive nécessite d'explorer $400 = 20 \times 20$ points.

Exemple (2/4) : Recherche aléatoire (Random Search)

Trajectoire de recherche aléatoire (50 étapes)

En bleu, le point initial, en gris le second max et en noir le max global.

→ Une recherche par marche aléatoire trouve, par chance, le second minimum. En 50 étapes d'exploration, seule une petite portion de S est explorée car les mêmes points sont souvent ré-explorés Inefficace !

Exemple (3/4) : Recherche Tabu (TS)

Trajectoire Tabu (50 étapes) avec taille de mémoire tabu = 4

Trajectoire Tabu (50 étapes) avec taille de mémoire tabu = 10

Avec une liste tabu de taille 4, la recherche se déplace autour du second maximum car elle oublie qu'il a déjà été visité. Avec une mémoire de taille 10, la recherche doit s'éloigner de cet extremum local et entrer dans le bassin d'attraction de l'extremum global. La trajectoire finit par se déplacer autour du max avec un chemin de longueur 10. La recherche est réussie !

Exemple (4/4) : Recherche Tabu (TS)

Trajectoire Tabu (167 étapes) avec taille de mémoire tabu = 200

Si la taille de la liste tabu continue d'augmenter, on visite de manière plutôt exhaustive l'espace de recherche. Mais on peut se retrouver bloqué (point rouge, atteint après 167 étapes).

Convergence de TS (1/3)

Convergence du processus de recherche : plusieurs définitions possibles.

TS trouve-t-elle l'optimum global dans un paysage de fitness donné ?

TS trouve-t-elle une solution “suffisamment bonne” dans un paysage de fitness donné selon un critère donné ?

La convergence dépend évidemment d'un certain nombre de facteurs : principalement

- Longueur de la mémoire (i.e., taille de la liste tabu)
- Taille/structure du voisinage
- Condition initiale
- Critère d'arrêt (s'il y en a un)

et bien sûr, elle dépend du paysage de fitness !

Convergence de TS (2/3)

Pour TS, nous avons le résultat mathématique suivant :

SI l'espace de recherche S est fini, et

SI le voisinage est symétrique ($s \in V(t) \Rightarrow t \in V(s)$), et

SI tout $s' \in S$ est atteignable depuis tout $s \in S$ en un nombre fini d'étapes

ALORS une TS qui stocke tous les points visités, et qui est également autorisée à revisiter le point le plus ancien de la liste, visitera tout l'espace de recherche, et trouvera donc l'optimum avec certitude.

Ce résultat mathématique a peu de pertinence pratique car :

- Il ne dit rien sur le temps nécessaire pour une telle exploration (il prédit un temps de recherche exponentiel étant donné la taille de S)
- Il nécessite une mémoire arbitrairement longue pour stocker la liste tabu

Convergence de TS (3/3) : Illustration

Trajectoire Tabu (400 étapes)

Trajectoire Tabu (600 étapes)

Lorsque la recherche se bloque, elle peut redémarrer depuis le point tabu le plus ancien. Cela peut prendre plus d'étapes que la taille de l'espace car de nombreux points tabu ont tous leurs voisins également tabu. On a souvent besoin de visiter une grande partie de la liste tabu avant de trouver une "sortie".

Liste Tabu (a.k.a. Liste d'Exclusion)

Objectifs et construction

Objectifs :

- Éviter de ré-échantillonner des configurations précédemment visitées
- Améliorer l'efficacité du processus d'exploration de S

Comment construire la liste tabu ? Que faut-il y inclure ?

- Configurations précédemment visitées
- Attributs des configurations précédemment visitées (par exemple, fitness, etc.)
- Souvent, **mouvements interdits** m_j parmi ceux définissant le voisinage

$$V(s) = \{s' \in S \mid s' = m_i(s), i = 1, \dots, \ell\}$$

où les mouvements m_i sont donnés

- Ou mouvements précédemment utilisés pour explorer l'espace de recherche

Plus sur les mouvements interdits (1/2)

À inclure dans la liste tabu :

→ **Mouvement inverse** d'un mouvement précédemment effectué.

Pour une transposition (i, j) , la même transposition est interdite (puisque'elle est son propre inverse).

Pour un marcheur 2D discret dans $\mathbb{Z}^2$ avec voisinage NEWS :

→ Après un mouvement **N**, on interdit un mouvement **S**

→ Après un mouvement **E**, on interdit un mouvement **W**

De telles stratégies empêchent les cycles simples qui ont le potentiel de bloquer le processus de recherche.

Plus sur les mouvements interdits (2/2)

Pourquoi les mouvements interdits sont-ils importants ?

- **Prévenir les cycles** : éviter les mouvements triviaux d'aller-retour
- **Encourager l'exploration** : force l'algorithme à considérer des mouvements non améliorants, l'aidant à échapper aux minima locaux
- **Équilibre exploitation vs. exploration** : le mécanisme tabu maintient une mémoire de l'activité récente, poussant la recherche vers des régions inexplorées

Mise à jour et gestion de la liste tabu

Exigence stricte :

→ La liste tabu ne peut pas être permanente.

Des mécanismes sont nécessaires pour effacer/vider les éléments tabu.

Deux types de mémoire peuvent être considérés :

→ **Mémoire à court terme** (Short-term memory)

→ **Mémoire à long terme** (Long-term memory)

Mémoire à court terme de taille limitée M

La mémoire à court terme obéit à l'une des deux approches suivantes :

1. Utiliser une liste tabu de taille finie et ne conserver que les exclusions les plus récentes (**FIFO** : First In, First Out)
2. Associer aux données dans la liste tabu une durée d'exclusion (temps d'interdiction k)

Seuls les derniers M éléments tabu sont conservés (les plus récents).

Mémoire à court terme avec temps d'interdiction k

Approche la plus courante :

La mémoire à court terme est une **map** avec les mouvements comme entrées, et la valeur associée est le temps pendant lequel ce mouvement est tabu.

Les mouvements tabu sont conservés pendant k itérations : k est le **temps d'interdiction** ou **tenure time**, ou **tabu tenure**.

La valeur optimale du temps d'interdiction k n'est pas connue :

→ Paramètre de guidage

→ Il est typiquement choisi aléatoirement avec une distribution de probabilité donnée dont les paramètres sont déterminés empiriquement.

Exemple de mémoire à court terme (1/2)

La liste tabu est :

- Construite comme un tableau T de taille M
- Construite sur un attribut $h(x)$ des solutions $x \in S$ ($h(x)$ est une fonction entière positive)

Soit x la solution à l'itération t . Nous stockons :

$$t + k \rightarrow T[h(x) \mod M]$$

(k étant le temps d'interdiction)

La configuration x' sera tabu à l'itération t' si :

$$t' < T[h(x') \mod M]$$

→ Avec ce mécanisme, les points ayant l'attribut $h(x')$ seront interdits pendant les k prochaines itérations.

Question importante : quelles sont les valeurs optimales de M et k ?

Voir le cas de la liste tabu pour le problème QAP.

Exemple de mémoire à court terme (2/2)

Marcheur aléatoire dans $\mathbb{Z}^2$ avec voisinage défini par les 4 mouvements NEWS : North, East, South et West.

Exemple de liste tabu (initialisée avec des zéros partout)

Mouvement	Tenure time(mémoire à court terme)	Fréquence de mouvement(mémoire à long terme)
North	0	0.5
East	0	0.5
South	3	0
West	5	0

À la première itération, $t = 1$, **North** est sélectionné pour se déplacer le long de la trajectoire exploratoire. Le mouvement inverse **South** devient donc tabu (pour éviter de revenir en arrière), disons pour $k = 2$ itérations. On met $t + k = 3$ dans la colonne “tenure time” pour l’entrée South (mouvement interdit pour les itérations #2 et #3).

À la seconde itération, $t = 2$, le mouvement **East** est sélectionné avec $k = 3 \rightarrow t + k = 5$ dans “tenure time” pour West (mouvement interdit pour les itérations #3 à #5).

Avec un grand tenure time k : exploration biaisée vers North et East.

→ D’où le besoin d’une mémoire à long terme.

Mémoire à long terme (Long-term Memory)

Observation :

→ Utiliser le même mouvement de manière répétée peut être acceptable à court terme, mais pas à long terme.

Remède :

→ Il est essentiel d’éviter les biais dans les sélections de mouvements.

Implémentation :

- Collecter des informations sur la trajectoire d’exploration complète depuis le début
- Générer des statistiques sur les mouvements/attributs/solutions passés
- Pénaliser les mouvements ayant une fréquence trop élevée, pour favoriser les moins fréquents (promouvoir l’exploration)
- Ce système bonus-malus peut être encore modulé par la valeur de la fitness (équilibre exploration vs. exploitation)

Utilisation de la mémoire

On peut combiner et/ou alterner l'utilisation des mémoires à court et long terme.

Il est nécessaire d'introduire des **paramètres de contrôle** pour les deux types de mémoire.

→ Paramètres de guidage

Encore une fois, les valeurs optimales pour ces paramètres de guidage sont inconnues (inconvenient des métaheuristiques).

Ces paramètres de contrôle de la mémoire doivent être ajoutés à d'autres paramètres de guidage inconnus : M et k .

Exploration vs. Exploitation

Le nombre d'entrées tabu ou de mouvements tabu influence le processus d'exploration.

Plus le nombre de mouvements interdits est grand, plus on choisit comme voisins des solutions avec une fitness plus faible.

→ **exploration** ou **diversification**

Si ce nombre est petit, on sera libre d'utiliser des solutions à haute fitness.

→ **exploitation** ou **intensification**

Un autre ingrédient pourrait être intégré, le soi-disant “**aspiration**” : si un point dans le voisinage a une fitness supérieure à celles rencontrées auparavant, ce point est sélectionné, même si le mouvement est interdit (tabu contourné).

Problème d'Affectation Quadratique (QAP)

Définition du QAP (1/2)

Appartient à une classe importante et difficile de problèmes d'optimisation combinatoire. **Étant donné :**

- n objets et n emplacements possibles pour ces objets
- Distances d_{rs} entre les emplacements r et s
- Poids ou flux f_{ij} entre les paires (i, j) d'objets

Objectif :

Trouver le placement optimal $i \rightarrow r_i$ pour les n objets aux n emplacements (= permutation) de manière à minimiser la fitness suivante :

$$f = \sum_{i,j} f_{ij} d_{r_i r_j} = \sum_{r,s} f_{i_r i_s} d_{rs}$$

où r_i dénote la position de l'objet i . Une autre notation courante est i_s dénotant l'objet à l'emplacement s .

L'espace de recherche S est donc celui des permutations de n objets.

Définition du QAP (2/2)

Le problème QAP consiste à trouver l'affectation optimale d'objets à des emplacements, en tenant compte à la fois des flux entre objets et des distances entre emplacements.

Exemples d'applications :

- Placement de composants électroniques sur un circuit
- Agencement d'installations industrielles
- Organisation spatiale de services dans un hôpital
- Tout problème où l'on doit minimiser les "coûts d'interaction" entre entités placées à différents emplacements

Exemple : Minimiser la longueur des connexions

n composants électroniques i (ou modules) qui doivent être affectés aux nœuds d'un réseau régulier.

Distances de Manhattan entre les nœuds d_{rs}

f_{ij} : nombre de fils/connexions entre i et j

Cela pourrait également représenter le flux de personnes entre les bâtiments.

Autre exemple : problème du voyageur de commerce (TSP)

→ Les emplacements sont les villes à visiter, les objets sont l'ordre de visite, et les flux sont booléens $f_{i,i+1} = 1$ ou 0.

Exemple avec 4 objets et 4 emplacements

Distances : $d_{AB} = 4$, $d_{AC} = 3$, $d_{AD} = 5$, $d_{BC} = 5$, $d_{BD} = 3$, $d_{CD} = 4$

Configuration de gauche :

Objets : 3, 2, 0, 1 aux emplacements A, B, C, D

La configuration de gauche a la fitness suivante :

$$\begin{aligned} f &= \sum_{i,j} f_{ij} d_{r_i r_j} \\ &= f_{01} d_{CD} + f_{03} d_{CA} + f_{13} d_{DA} + f_{23} d_{BA} \\ &= 1 \times 4 + 1 \times 3 + 2 \times 5 + 3 \times 4 = 29 \end{aligned}$$

Et pour la configuration de droite :

Objets : 3, 1, 2, 0 aux emplacements A, B, C, D

$$\begin{aligned} f &= f_{01} d_{DB} + f_{03} d_{DA} + f_{13} d_{BA} + f_{23} d_{CA} \\ &= 1 \times 3 + 1 \times 5 + 2 \times 4 + 3 \times 3 = 25 \end{aligned}$$

Matrices de flux et de distance

Matrice de flux :

$$\text{Flow} = \begin{pmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 2 \\ 0 & 0 & 0 & 3 \\ 1 & 2 & 3 & 0 \end{pmatrix}$$

Matrice de distance :

$$\text{Distance} = \begin{pmatrix} 0 & 4 & 3 & 5 \\ 4 & 0 & 5 & 3 \\ 3 & 5 & 0 & 4 \\ 5 & 3 & 4 & 0 \end{pmatrix}$$

Code pour calculer la fitness QAP

```
def QAP_Fitness(permutation, Flow, Dist):
    n = len(F)
    fitness = 0
    for r in range(n):
        for s in range(r, n):
            fitness += Dist[r][s] * Flow[permutation[r]][permutation[s]]
    return fitness

permutation = [1, 3, 0, 2]
# permutation[r] = objet placé à l'emplacement r
```

Emplacements A, B, C, D - Exemples de fitness :

- [0, 1, 2, 3] fitness = 27
- [1, 0, 2, 3] fitness = 29
- [2, 0, 1, 3] fitness = 31
- [0, 2, 1, 3] fitness = 25
- [1, 2, 0, 3] fitness = 26
- [2, 1, 0, 3] fitness = 30
- [3, 1, 0, 2] fitness = 31
- [1, 3, 0, 2] fitness = 25
- [0, 3, 1, 2] fitness = 26
- [3, 0, 1, 2] fitness = 30
- [1, 0, 3, 2] fitness = 27
- [0, 1, 3, 2] fitness = 29
- [0, 2, 3, 1] fitness = 31
- [2, 0, 3, 1] fitness = 25
- [3, 0, 2, 1] fitness = 26
- [0, 3, 2, 1] fitness = 30
- [2, 3, 0, 1] fitness = 27
- [3, 2, 0, 1] fitness = 29
- [3, 2, 1, 0] fitness = 27
- [2, 3, 1, 0] fitness = 29
- [1, 3, 2, 0] fitness = 31
- [3, 1, 2, 0] fitness = 25
- [2, 1, 3, 0] fitness = 26
- [1, 2, 3, 0] fitness = 30

Solution par Tabu Search

Espace de recherche : toutes les permutations

$$p = (i_1, i_2, \dots, i_n)$$

où i_k est l'indice de l'objet à l'emplacement k .

→ Alternativement, on pourrait prendre $p = (r_1, r_2, \dots, r_n)$ où r_i est l'emplacement sélectionné pour l'objet i .

Voisinage : choisir les transformations/mouvements le construisant.

Voisinages standards

Échanger deux objets contigus :

$$(i_1, i_2, \dots, i_k, i_{k+1}, \dots, i_n) \rightarrow (i_1, i_2, \dots, i_{k+1}, i_k, \dots, i_n)$$

$n - 1$ mouvements possibles dans ce voisinage.

Échanger deux objets quelconques :

$$(i_1, i_2, \dots, i_k, \dots, i_\ell, \dots, i_n) \rightarrow (i_1, i_2, \dots, i_\ell, \dots, i_k, \dots, i_n)$$

$n(n - 1)/2$ mouvements possibles dans ce voisinage.

En déplaçant et insérant un objet :

$$(i_1, i_2, \dots, i_k, i_{k+1}, \dots, i_\ell, i_{\ell+1}, \dots, i_n) \rightarrow (i_1, i_2, \dots, i_k, i_\ell, i_{k+1}, \dots, i_{\ell-1}, i_{\ell+1}, \dots, i_n)$$

On peut montrer que ce voisinage admet $n(n - 2) + 1$ voisins.

Évaluation du voisinage

Plus le nombre de mouvements m_i est important, plus il faudra de temps pour évaluer la fitness de tous les voisins.

→ Voisinage plus grand = exploration plus élevée

À partir de la configuration courante p , nous devons calculer :

$$\Delta_i = f(m_i(p)) - f(p)$$

→ Pour un processus de minimisation, nous cherchons $\Delta_i < 0$ dans le voisinage.

En pratique, nous choisissons entre les stratégies d'échange et d'insertion : $|V| \in O(n^2)$.

Si seulement 2 objets sont déplacés, la plupart des termes dans la fitness n'ont pas besoin d'être modifiés.

Seulement $O(n)$ termes doivent être calculés pour obtenir Δ_i .

Liste tabu et mémoire à court terme

Le mouvement dénoté par (r, s) correspond à l'échange d'objets aux emplacements r et s :

$$(\dots, i_r, \dots, i_s, \dots) \xrightarrow{(r,s)} (\dots, i_s, \dots, i_r, \dots)$$

Si, par le mouvement sélectionné, l'objet i a quitté l'emplacement r et l'objet j a quitté l'emplacement s , alors il est interdit de replacer i en r **ET** j en s pendant les t prochaines itérations.

Implémentation de la liste tabu (1/2)

La liste tabu est représentée comme une **matrice carrée** $n \times n$ T .

L'entrée T_{ir} correspond à l'itération à laquelle l'objet i a quitté l'emplacement r pour la dernière fois, à laquelle on ajoute le temps d'interdiction k .

Le mouvement (r, s) est interdit si $T_{i_s r}$ et $T_{i_r s}$ contiennent tous deux une valeur supérieure au compteur d'itération actuel.

Ceci est un exemple de liste tabu basée sur les **attributs de la solution**.

Implémentation de la liste tabu (2/2)

En pratique :

Nous tirons uniformément et aléatoirement le temps d'interdiction k dans l'intervalle :

$$k \in [0.9 \times n, 1.1 \times n + 4]$$

(les bornes sont sélectionnées empiriquement et correspondent au réglage de la métaheuristique. La valeur k ne peut pas être trop grande sinon elle bloquerait tous les mouvements).

Le problème de référence NUG5

NUG5 consiste en un problème de référence (**benchmark**) pour la classe QAP.

C'est un problème de taille $n = 5$ (d'où son nom) défini par la matrice de flux f_{ij} et la matrice de distance d_{rs} suivantes :

$$F = \begin{pmatrix} 0 & 5 & 2 & 4 & 1 \\ 5 & 0 & 3 & 0 & 2 \\ 2 & 3 & 0 & 0 & 0 \\ 4 & 0 & 0 & 0 & 5 \\ 1 & 2 & 0 & 5 & 0 \end{pmatrix} \quad D = \begin{pmatrix} 0 & 1 & 1 & 2 & 3 \\ 1 & 0 & 2 & 1 & 2 \\ 1 & 2 & 0 & 1 & 2 \\ 2 & 1 & 1 & 0 & 1 \\ 3 & 2 & 2 & 1 & 0 \end{pmatrix}$$

Problème de référence conçu de sorte que :

- Le problème a un **optimum global unique**
- Il est suffisamment petit pour être résolu exactement (utilisé pour la validation)
- Il capture la structure quadratique typique des problèmes de type affectation

Application de Tabu Search sur NUG5

Solution initiale représentée par la permutation :

$$p_1 = (2, 4, 1, 5, 3)$$

→ dont la fitness $f(p_1) = 72$ peut être facilement calculée à partir des matrices F et D :

$$f = \sum_{r,s} f_{i_r, i_s} d_{rs}$$

La liste tabu est initialisée avec des zéros : $T_{ir} = 0, \forall i, r$.

Itération #1

m_i	(1,2)	(1,3)	(1,4)	(1,5)	(2,3)	(2,4)	(2,5)	(3,4)	(3,5)	(4,5)
Δ_i	2	-12	-12	2	0	-10	-12	4	8	6

Nous remarquons que les 3 mouvements (1,3), (1,4) et (2,5) donnent la meilleure réduction de la fitness.

L'un de ces 3 mouvements est choisi aléatoirement, par exemple :

$$m_2 = (1, 3)$$

Étant donné la solution courante $p_1 = (2, 4, 1, 5, 3)$, le mouvement m_2 donne la nouvelle configuration suivante :

$$p_2 = (1, 4, 2, 5, 3)$$

avec fitness $f(p_2) = f(p_1) + \Delta = 72 - 12 = 60$.

Liste tabu à l'itération #1 (1/2)

Le mouvement sélectionné $m_2 = (1, 3)$ a fait que l'objet 1 quitte la position 3 et l'objet 2 quitte la position 1 :

→ Les entrées T_{13} et T_{21} de la matrice tabu doivent être mises à jour.

Nous tirons uniformément et aléatoirement le temps d'interdiction k dans l'intervalle :

$$k \in [0.9 \times n, 1.1 \times n + 4]$$

La matrice tabu admet $n \times n$ entrées, mais à chaque itération seules deux entrées sont ajoutées/mises à jour.

Liste tabu à l'itération #1 (2/2)

Supposons que nous tirions aléatoirement $k = 9$. Puisque nous sommes à l'itération #1, le tabu de remplacer l'objet 1 (resp. 2) à l'emplacement 3 (resp. 1) durera jusqu'à l'itération $1 + k = 10$.

La matrice tabu mise à jour est :

$$T = \begin{pmatrix} 0 & 0 & 10 & 0 & 0 \\ 10 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

Itération #2

Nous avons maintenant le tableau suivant pour tous les mouvements possibles :

m_i	(1,2)	(1,3)	(1,4)	(1,5)	(2,3)	(2,4)	(2,5)	(3,4)	(3,5)	(4,5)
Δ_i	14	12	-8	10	0	10	8	12	12	6

Le mouvement (1,3) est **tabu** car c'est l'inverse du mouvement précédemment accepté !

Le mouvement $m_3 = (1,4)$ donne la plus grande réduction de fitness, donc il est sélectionné.

Ce mouvement donne la nouvelle configuration suivante :

$$p_3 = (5, 4, 2, 1, 3)$$

avec fitness $f(p_3) = 60 - 8 = 52$.

Liste tabu à l'itération #2

Le mouvement $m_3 = (1,4)$ affecte les entrées T_{11} (objet 1 à l'emplacement 1) et T_{54} (objet 5 à l'emplacement 4).

En supposant un tirage aléatoire de $k = 6$, le temps d'interdiction s'étend jusqu'à l'itération $2 + 6 = 8$, et nous avons :

$$T = \begin{pmatrix} 8 & 0 & 10 & 0 & 0 \\ 10 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 8 & 0 \end{pmatrix}$$

Itération #3

Nous avons maintenant le tableau suivant pour tous les mouvements possibles :

m_i	(1,2)	(1,3)	(1,4)	(1,5)	(2,3)	(2,4)	(2,5)	(3,4)	(3,5)	(4,5)
Δ_i	24	10	8	10	0	22	20	8	8	14

À ce stade, le mouvement (1,4) est tabu mais plus (1,3) qui déplacerait l'objet 5 en position 3, ce qui est possible puisque $T_{53} = 0$ (l'objet 5 n'a jamais été en position 3).

Le mouvement $m_5 = (2,3)$ est le mouvement à coût minimal bien qu'il n'améliore pas la fitness étant donné $\Delta_5 = 0$. C'est néanmoins celui sélectionné pour générer la configuration suivante :

$$p_4 = (5, 2, 4, 1, 3)$$

avec $f(p_4) = 52$.

Liste tabu à l'itération #3

Le mouvement $m_5 = (2,3)$ affecte les entrées T_{23} (objet 2 quittant l'emplacement 3) et T_{42} (objet 4 a quitté l'emplacement 2).

En supposant un tirage aléatoire de $k = 8$, le temps d'interdiction s'étend jusqu'à l'itération $3 + 8 = 11$, et nous avons :

$$T = \begin{pmatrix} 8 & 0 & 10 & 0 & 0 \\ 10 & 0 & 11 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 11 & 0 & 0 & 0 \\ 0 & 0 & 0 & 8 & 0 \end{pmatrix}$$

Itération #4

Nous avons maintenant le tableau suivant pour tous les mouvements possibles :

m_i	(1,2)	(1,3)	(1,4)	(1,5)	(2,3)	(2,4)	(2,5)	(3,4)	(3,5)	(4,5)
Δ_i	24	10	8	10	0	8	8	22	20	14

Nous avons maintenant 2 mouvements tabu (en italique ci-dessus) : $m_3 = (1, 4)$ et $m_5 = (2, 3)$.

- Pour le mouvement m_5 , ce n'est pas surprenant car il inverserait le mouvement précédent et ramènerait les objets dans leur configuration précédente
- Pour le mouvement $m_3 = (1, 4)$, il est encore interdit car il ramènerait les objets 5 et 1 dans des configurations précédemment occupées

Les mouvements "optimaux" autorisés sont $m_6 = (2, 4)$ et $m_7 = (2, 5)$, tous deux donnant une dégradation de la fitness de 8 unités pendant cette itération.

Nous supposons que $m_6 = (2, 4)$ est sélectionné comme prochain mouvement donnant la configuration suivante :

$$p_5 = (5, 1, 4, 2, 3)$$

avec fitness $f(p_5) = 60 = 52 + 8$.

Liste tabu à l'itération #4

Le mouvement $m_6 = (2, 4)$ affecte les entrées T_{22} (objet 2 quittant l'emplacement 2) et T_{14} (objet 1 a quitté l'emplacement 4).

En supposant un tirage aléatoire de $k = 5$, le temps d'interdiction s'étend jusqu'à l'itération $4 + 5 = 9$, et nous avons :

$$T = \begin{pmatrix} 8 & 0 & 10 & 9 & 0 \\ 10 & 9 & 11 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 11 & 0 & 0 & 0 \\ 0 & 0 & 0 & 8 & 0 \end{pmatrix}$$

Itérations suivantes

Ce processus est itéré jusqu'à ce qu'un critère d'arrêt donné soit atteint.

À l'itération #5, vous pouvez vérifier que le mouvement sélectionné est (1,3) donnant une réduction de la fitness de 10 unités et avec la nouvelle configuration suivante :

$$p_6 = (4, 1, 5, 2, 3)$$

avec fitness $f(p_6) = 50$.

Cela en fait la **solution optimale jusqu'à présent**, ce qui montre qu'accepter une dégradation de fitness à l'itération précédente a effectivement été bénéfique pour atteindre de “meilleures” régions de l'espace de recherche.